

A Near-Data Processing Approach to Accelerating Compaction in LSM-tree Key-Value Stores

Jaesik Jang
Yonsei University
Seoul, Republic of Korea
jseee@yonsei.ac.kr

Seunghyun Lee
Yonsei University
Seoul, Republic of Korea
hyunof23@yonsei.ac.kr

Abstract

Log-Structured Merge (LSM) tree-based key-value stores have become indispensable for high-performance read/write operations; however, they suffer from significant performance degradation due to unavoidable background compaction. This process, which relies on the host CPU to execute resource-heavy merge sort operations, frequently saturates the limited bandwidth of the DDR bus and leads to severe cache pollution. To alleviate this data movement bottleneck, this work introduces a Processing-Near-Memory (PNM) architecture designed to handle compaction directly within the DIMM module. By embedding dedicated Near-Memory Processing (NMP) Unit into the buffer chips, our design facilitates local data merging, effectively utilizing vast internal bandwidth without necessitating host CPU intervention. Furthermore, by employing a cache-bypassing DMA mechanism, this architecture fundamentally prevents cache pollution. Detailed cycle-level evaluations reveal that our PNM architecture successfully reduces L2 cache misses to 35.7% of the baseline and increases effective memory bandwidth by 1.49x. Consequently, the total execution time is reduced to 73.5% of the baseline. Ultimately, this study offers a practical hardware-accelerated solution that maximizes the throughput of next-generation database systems by shifting toward a memory-centric computing paradigm.

Keywords: LSM-tree, Compaction, Near-Data Processing, Processing-Near-Memory

1. Introduction

Current database systems often rely on the Log-Structured Merge (LSM) tree [17] to maintain high write throughput for data-intensive applications (e.g., RocksDB [4, 5], LevelDB [7, 14]). To achieve this, random write requests are gathered in a memory-resident Memory Table (MemTable)

and subsequently flushed to disk as immutable Sorted-String Tables (SSTables) when the buffer reaches its capacity. Overtime, SSTables include overlapping key ranges, necessitating a background compaction operation to consolidate data and optimize read performance. This compaction requires the system to load multiple SSTables into the main memory and perform a merge sort. In conventional systems, the host CPU is responsible for these sorting operations, forcing massive volumes of data to be transferred across the limited DDR bus. This creates a severe bandwidth bottleneck and leads to cache pollution, as the large-scale data movement for background sorting evicts critical query data from the processor’s cache, thereby degrading the performance of foreground user queries.

To resolve the fundamental data movement bottleneck, we propose a DIMM-based Processing-Near-Memory (PNM) architecture that executes compaction operations directly within the memory hierarchy. The proposed design integrates a dedicated Near-Memory Processing (NMP) Unit into the buffer chips of the DIMM module, allowing for a ‘processor-to-data’ paradigm. During compaction, the host CPU no longer needs to fetch raw data for comparison; instead, it simply writes control commands to the dedicated Memory-Mapped I/O (MMIO) registers. By performing the merge sort locally and returning the final result directly to memory, this architecture effectively isolates background compaction traffic from the main memory channel. This approach ensures that the host-memory bandwidth is preserved for time-sensitive application logic rather than being consumed by maintenance tasks.

We validated the effectiveness of the proposed PNM architecture through a cycle-level analysis using the gem5 [15] simulator. Our experimental results demonstrate that offloading compaction to the memory side significantly reduces execution time by eliminating redundant data transfers. By bypassing the CPU’s cache hierarchy during background compaction, the proposed PNM architecture effectively prevents cache pollution, reducing L2 cache misses to 35.7% of the baseline. This structural isolation reclaims

the host memory channel for foreground queries, increasing effective memory bandwidth by 1.49x and directly translating to a significant improvement in overall transaction throughput, where the total execution time is reduced to 73.5% of the baseline. Ultimately, this research presents a memory-centric solution that overcomes the performance degradation and write amplification issues inherent in traditional key-value stores, providing a scalable path for next-generation high-performance database systems.

2. Background

2.1. LSM-tree and Compaction

Modern data-intensive applications utilize LSM-trees to provide efficient indexed access while sustaining high insert volumes. By treating incoming data as sequential additions, the LSM-tree avoids costly in-place updates and random I/O, significantly boosting write throughput.

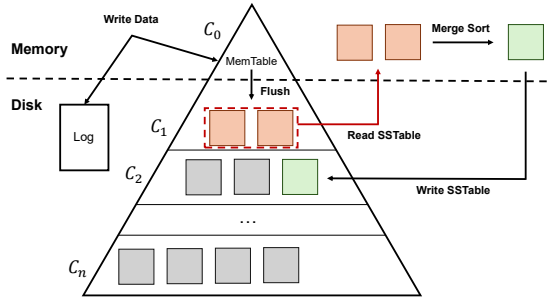


Figure 1. LSM-tree Structure and Compaction Procedure.

The LSM-tree maintains key-value pairs across a tiered hierarchy of a memory-resident component (C_0) and multiple disk-level components ($C_1, C_2, \dots, C_n$), as illustrated in Figure 1. For durability, new data is first appended to a Write-Ahead Log (WAL) on disk to ensure recovery after a crash. The data is then stored in an active MemTable within C_0 , which maintains a sorted order. Once the active MemTable reaches a size threshold, it is frozen into an immutable MemTable. A background thread flushes the immutable MemTable to C_1 as an SSTable.

Because each SSTable contains specific key ranges, the system performs compaction, depicted as movement between levels in Figure 1, to optimize read performance and search bounds. When the number of SSTables in a level reaches a threshold, the system merges multiple SSTables, sorting the streams into a single output while purging obsolete or overlapping keys. The outcome is then placed at a lower level.

While essential, compaction is the primary source of overhead in LSM-trees. Data is repeatedly read, sorted, and re-written, leading to significant I/O amplification. Fur-

thermore, the CPU must manage this logic while simultaneously handling foreground threads; this constant data movement across storage, memory, and cache often saturates memory bandwidth for what is ultimately a computationally simple task.

2.2. Near-Data Processing

In contemporary data-intensive environments (e.g., big data processing [16, 19], machine learning [11, 22]), the continuous data movement between the host CPU and memory devices creates a severe bandwidth bottleneck. The Near-Data Processing paradigm addresses this fundamental limitation by moving compute capabilities close to the memory, enabling memory-centric computing and minimizing unnecessary data transfers. For bandwidth-bound workloads like LSM-tree compaction, a PNM architecture that places compute units directly within the main memory hierarchy is highly effective.

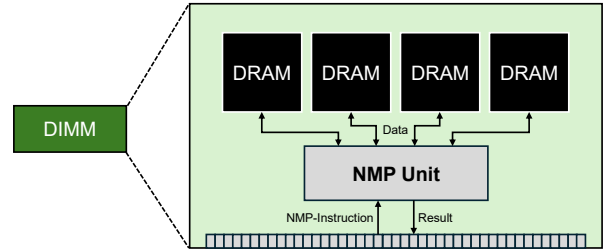


Figure 2. DIMM-based PNM.

Among various PNM designs, DIMM-based PNM offers a practical and scalable solution that maintains standard memory channel interfaces [12], [13]. As illustrated in Figure 2, by integrating an NMP Unit into the DIMM's buffer chips, it requires no modifications to commercial off-the-shelf DRAM components. This structure leverages rank-level parallelism, allowing the simultaneous activation of multiple ranks to dramatically expand internal memory bandwidth.

When applied to LSM-tree compaction, the host CPU simply dispatches control commands (NMP-instructions) instead of fetching massive amounts of data. The NMP Unit reads data across multiple ranks in parallel, performs necessary comparison and merge operations locally, and returns only the completion signal to the host. This drastically reduces host memory channel traffic and accelerates heavy sorting operations without host CPU intervention.

3. PNM Architecture

3.1. System Architecture

The PNM architecture proposed in this study is designed to achieve structural isolation between foreground query processing and background compaction operations. The over-

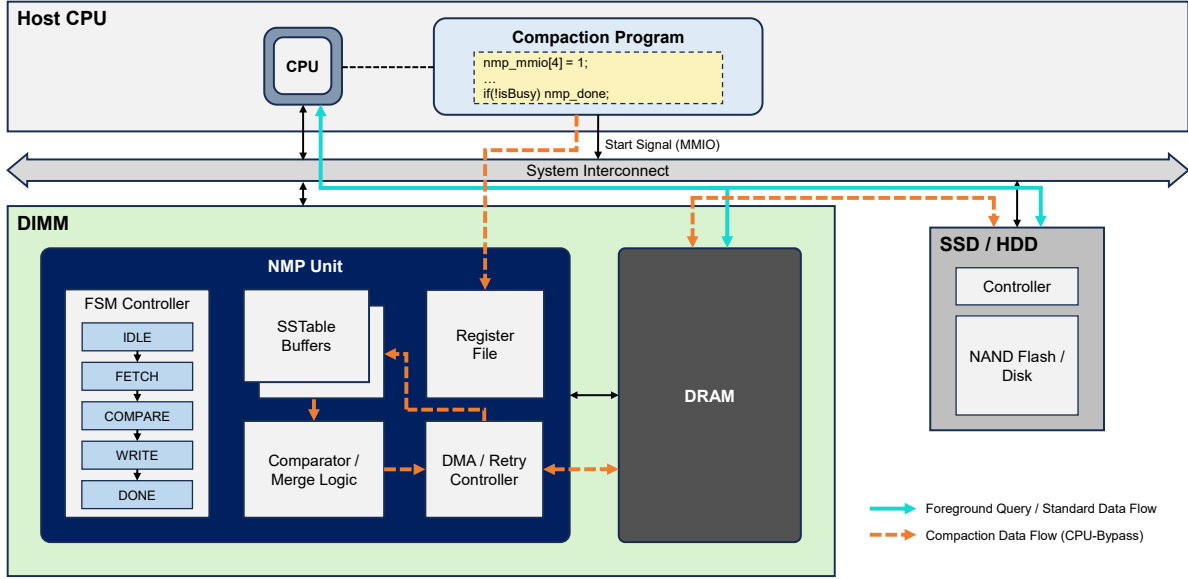


Figure 3. System Architecture and Data Flow.

all hardware topology of the system consists of a host CPU, a system interconnect, a DIMM integrating DRAM and the NMP Unit accelerator, and storage devices (SSD/HDD) (see Figure 3). The NMP Unit, the core of this architecture, is connected in parallel to the shared system bus as an independent PioDevice and RequestPort over a general-purpose bus topology. Consequently, the host CPU's standard key-value point lookup and update query traffic contends with the accelerator's background compaction traffic over the same interconnect structure. This study proposes an architecture that arbitrates this contention at the hardware level.

3.2. Register File

Communication and the transfer of control authority between the user program running on the host system and the NMP Unit are physically executed through MMIO. This architecture defines a dedicated register file mapped to a specific physical address space for the configuration and control of the NMP Unit. The address offset structure of the register file is subdivided into two input base registers specifying the starting physical addresses of the two source SSTables targeted for compaction, an output base register designating the destination address where the sorted data will be written, and a counter register indicating the total number of items held by each table. A control register serves as a start trigger. The interface logic circuit detects it and drives the merge engine. The completion status of the compaction operation is synchronized via a status register.

3.3. FSM Controller

The internal control path of the NMP Unit is governed by a Finite State Machine (FSM) controller that asynchronously schedules the entire merge process. The FSM encompasses five distinct physical states: IDLE, FETCH, COMPARE, WRITE, and DONE.

- **IDLE:** The NMP Unit remains in a standby state.
- **FETCH:** The controller searches for an empty slot between the two source SStable buffers and requests a new key-value record from the corresponding DRAM physical address.
- **COMPARE:** All buffers are loaded with valid key values. A hardware comparator selects the minimum value and achieves deduplication.
- **WRITE:** The selected minimum value is flushed to the DRAM's output buffer region via the system bus.
- **DONE:** Merge sort and output creation are complete. The status register is deasserted to 0, returning the FSM to the IDLE state.

3.4. Microarchitecture Components

The data path within the NMP Unit is meticulously constructed entirely of pure hardware components to minimize area overhead and reduce latency. The core components of this hardware data path include the Address Generation Unit (AGU), SStable Buffers, and 2-Way Comparator Logic. When the NMP Unit fetches a specific memory block, the embedded AGU immediately performs an address translation arithmetic operation at the hardware level, multiplying the current item index by the physical record size of 1040 bytes (16-byte Key + 1024-byte Value region)

and adding it to the source base address. Data fetched from the main memory via DMA is loaded into the SSTable buffers. Instead of employing a bulky SRAM cache structure, these buffers are designed with an ultra-lightweight architecture consisting of two high-speed flip-flop registers holding 128-bit key values, 1-bit registers marking data validity, and a dedicated register to latch the last written key for deduplication filtering, thereby minimizing die area consumption.

Priority queues (Min-Heaps), commonly utilized in software algorithms, are unsuitable for hardware implementation due to significant pointer rearrangement and memory overhead [3]. Therefore, this architecture independently adopts a 2-Way 128-bit hardware comparator logic circuit. This architecture simultaneously evaluates the two key values loaded in the buffers, ensuring extremely fast computation by processing the absolute minimum value among the data within a single clock cycle. Furthermore, once the minimum key is determined, the logic immediately compares it against the previously latched key in the dedicated register. If a match is detected, the duplicate record is actively discarded and the memory write operation is bypassed, thereby fundamentally eliminating redundant memory write traffic.

3.5. DMA/Retry Controller

Under a Shared Bus Topology, the massive memory requests of background compaction inevitably induce conflicts with the host CPU's foreground query memory traffic. When the system interconnect reaches saturation, the memory controller returns a rejection response to the timing requests transmitted by the NMP Unit, which becomes a critical cause of packet loss and permanent deadlock within the FSM. To resolve this, a DMA/Retry Controller architecture is introduced within the memory-side port of the NMP Unit. This controller incorporates a pending packet pointer (pendingPkt) that safely latches and holds packets rejected due to bus contention within a physical register space, preventing them from being dropped. When the congestion of the system bus is alleviated and a retry request signal from the memory subsystem arrives at the NMP Unit port, the retry controller intervenes to execute a hardware protocol that re-emits the request stored in pendingPkt to the bus. Through this Retry mechanism, a robust asynchronous DMA data transmission pipeline with no data loss can be established, even amid fluctuating traffic on the main bus.

4. System Emulation Mode

Before evaluating the proposed architecture in Full System (FS) mode, we first conducted experiments using System Emulation (SE) mode. The primary purpose of this preliminary evaluation was to extract and measure the pure microarchitectural performance improvements and hardware specifications of the PNM architecture, strictly isolating

them from the interference of the OS and complex software stacks.

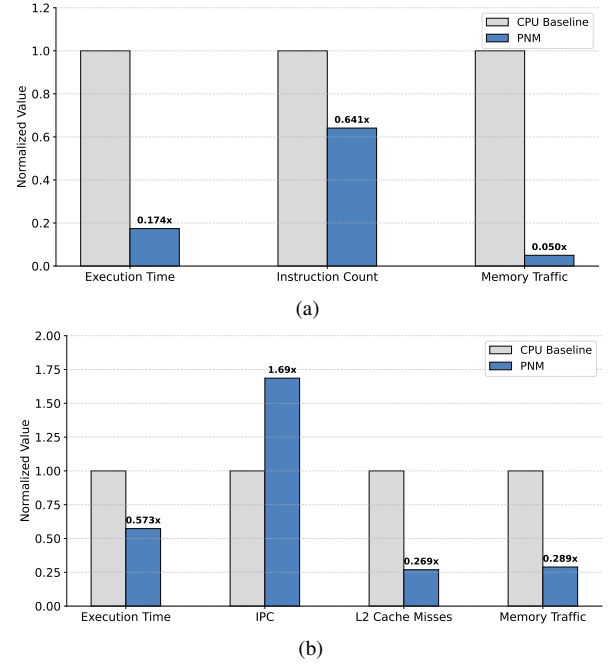


Figure 4. Comparison of performance and microarchitectural metrics between the CPU baseline and PNM under SE mode: (a) Executing a single compaction task; (b) Executing foreground database queries concurrently with background compaction.

Compaction-Only. Figure 4(a) illustrates the hardware metrics when executing a single compaction operation independently. The conventional CPU baseline must rely on heavy software control loops and memory pointer arithmetic to perform the merge sort algorithm. In contrast, the proposed PNM architecture replaces these operations with a dedicated hardware FSM and a 2-Way hardware comparator, thereby reducing the total instruction count by 35.9% compared to the baseline.

A particularly notable aspect is the change in total memory traffic. Through the hardware-level deduplication logic implemented within the accelerator and cache-bypassing DMA, unnecessary data movement and instruction fetch traffic are fundamentally blocked. As a result, memory traffic is drastically decreased to 5.0% of the baseline. By fundamentally eliminating these computational and memory overheads, the pure compaction execution time is reduced to 0.174x compared to the baseline, achieving approximately a 5.75x performance improvement.

Foreground Queries with Background Compaction. Figure 4(b) shows the metrics in an environment where latency-sensitive foreground database queries and background compaction are executed concurrently. This experiment clearly demonstrates the impact of large-scale data

movement on the system cache and memory bandwidth.

In the baseline environment where the CPU directly performs compaction, the compaction target data continuously overwrites the limited 512KiB L2 cache, inducing severe cache thrashing. Consequently, the system suffers from extreme cache misses during foreground query processing. In contrast, the proposed PNM architecture communicates by bypassing the cache, significantly lowering L2 cache misses to 26.9% of the baseline and successfully preventing cache pollution.

As the cache state is effectively preserved, the CPU can focus on foreground operations without being hindered by background compaction, leading to a 1.69x increase in the CPU’s IPC. Furthermore, as cascading memory requests due to unnecessary cache line evictions are eliminated and the hardware-level deduplication takes effect, the total memory traffic imposed on the main system bus is also reduced to 28.9% of the baseline. Consequently, the PNM architecture perfectly isolates the compaction overhead from the main system, successfully reducing the total execution time by 42.7%.

5. Full System Mode

5.1. Evaluation Methodology

Table 1. System Specifications

Processor	Single-Core CPU, 3GHz
L1 I/D Caches	32KiB
L2 Cache	512KiB
Main Memory	Dual-Channel DDR4-2400 (Interleaved) 3GiB

To comprehensively evaluate the performance and performance isolation effects of the hardware acceleration architecture proposed in this study, we constructed an experimental environment utilizing the FS mode of a cycle-accurate simulator. In the FS environment, the benchmark was executed on top of an actual operating system to model realistic system behavior.

Table 2. Workload Characteristics

Micro-Benchmark	db_bench (RocksDB)
Random Write Queries	25,000
Random Read Queries	25,000
MemTable/SSTable Size	1MiB

The workload used in this evaluation was designed based on the operational characteristics of db_bench [6] from RocksDB, a standard micro-benchmark for LSM-tree-based databases. To ensure a fair microarchitectural comparison

between the CPU baseline and the proposed PNM architecture, WAL and data compression options were explicitly disabled, thereby isolating the pure memory and computation overheads of the compaction operation. In the FS environment, db_bench was run directly on a Linux kernel to generate actual database server traffic, where foreground queries and background compactions were mixed. RocksDB deployed foreground queries and triggered background compaction over our PNM architecture, which was integrated into gem5 as a new hardware device model. A kernel driver to allow the guest OS to access the MMIO region was added, and the db_bench micro-benchmark was modified to route compaction jobs to the new device. Together, these additions modeled the full system behavior described in Section 3 under realistic OS conditions.

The overall experimental scenario was configured to execute 25,000 random write queries and 25,000 random read queries. Through this setup, we created an environment where latency-sensitive foreground query traffic and background traffic involving large-scale sequential memory accesses contend for limited system resources.

5.2. Evaluation

Figure 5 presents the performance metrics of RocksDB’s workload under full Linux OS conditions. The PNM architecture’s cache-bypassing DMA mechanism is particularly effective in the FS environment. Because the DMA mechanism was validated in SE mode to eliminate all compaction cache traffic, it operates through the same hardware data path under the full OS — the kernel and file system do not alter the DMA routing. L2 cache misses are reduced to 35.7% of the baseline. Unlike OS-level scheduling hints or CPU affinity policies, the DMA mechanism prevents compaction data from contaminating the L2 cache. This results in a 26.6% reduction in per-query access latency, as query-hot working sets remain resident across both the write and read benchmark phases.

The separation of traffic paths frees the foreground queries from competing with compaction’s sequential scan traffic on the system bus. The host memory channel reclaims capacity for foreground operations, yielding a 1.49x increase in effective memory bandwidth. This improvement, together with the latency drop, creates a compound benefit: cached data is hit more often, and the bus is less contended simultaneously. Total memory traffic is reduced to 89.5% of the baseline; the residual overhead is attributable to OS kernel activity and RocksDB’s background write path, which persist independently of the compaction architecture.

The NMP Unit’s FSM and 2-Way comparator circuit reduce total compaction time to 70.1% of the baseline. While software implementations under OS scheduling pressure incur instruction fetch and branch overhead per com-

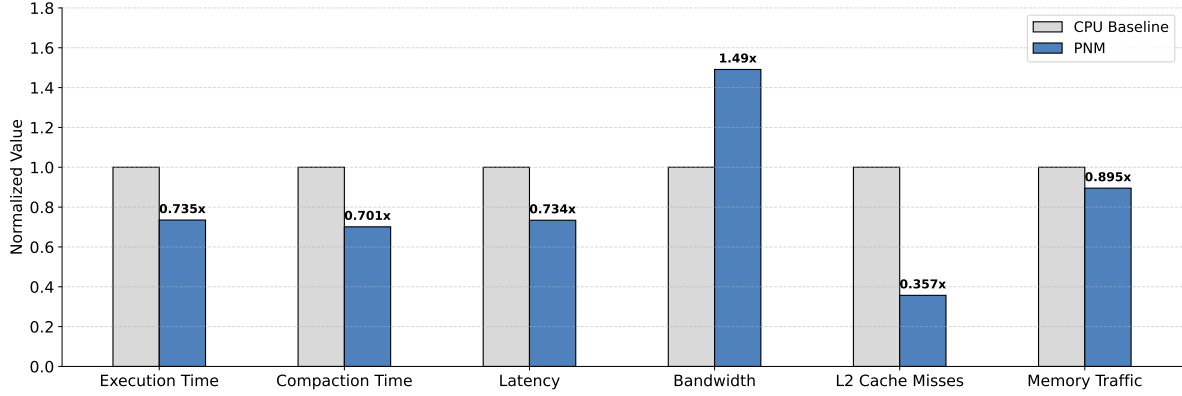


Figure 5. Comparison of performance and microarchitectural metrics between the CPU baseline and PNM under FS mode, executing foreground database queries concurrently with background compaction.

parison, having a dedicated hardware FSM advances compaction independently of foreground threads. The shortened compaction window proportionally narrows the duration over which background traffic occupies the shared interconnect, compounding the cache and bandwidth gains described above. Together, these improvements reduce total execution time to 73.5% of the baseline, demonstrating that the DIMM-based PNM architecture preserves its acceleration effect across the full complexity of a commercial software stack, even as OS scheduling and file system overhead moderate the gains relative to SE mode.

6. Discussion

SE versus FS. The experimental results from the SE mode and the FS mode exhibit different behavior depending on the presence of an OS and the degree of intervention from the overall system software stack. In the SE environment, foreground queries and background compaction were executed concurrently while excluding OS interference. This allowed us to measure the pure microarchitectural performance improvements and cache pollution prevention effects provided by the PNM accelerator, without software noise such as OS scheduling or interrupt handling. In contrast, the experiments in the FS environment were conducted on an actual Linux kernel and the file system I/O stack of RocksDB. Although the performance was measured to be lower than in the SE environment due to the realistic constraints of added OS context switching and background process overheads, the proposed PNM architecture successfully demonstrated its effectiveness in a commercial software environment by reducing unnecessary traffic and improving the overall system processing speed.

Limitations. The evaluation was conducted on a limited workload of approximately 50MiB, consisting of a combined total of 50,000 keys and values for both read and write operations. Compared to actual production environ-

ments that handle terabytes of data, the dataset size is relatively small; therefore, extreme multi-level compaction scenarios, where tens to hundreds of large-scale SSTables are merged, were not fully reflected in the experiment. Furthermore, to secure a rapid experimental cycle in a simulator environment that requires cycle-level accuracy, and to more clearly and intuitively observe the bottlenecks caused by compaction and the acceleration effects of PNM, the overall system specifications including the number of CPU cores, cache size, and memory capacity were intentionally scaled down. In an enterprise-grade, high-performance server environment equipped with massive resources, the patterns of resource contention and the acceleration efficiency may exhibit numerical differences from our experimental results.

Trade-offs. While this evaluation focused on improvements in performance and memory traffic metrics (e.g., Execution Time, Cache Misses, Bandwidth), physically implementing PNM within an actual DIMM module entails several hardware trade-offs. Since logic components such as the FSM Controller, Comparator, and DMA/Retry Controller inside the NMP Unit must be integrated into the DIMM’s buffer chips, the silicon area of the chips increases compared to standard DIMMs, leading to a cost trade-off involving higher design and manufacturing expenses. Additionally, although blocking the massive data traffic traveling to the central CPU can reduce the overall system I/O power consumption, operating the compute units directly within the memory module increases the power density and thermal dissipation in localized areas of the DIMM. This may necessitate the addition of new cooling solutions or precise thermal control logic within the standard JEDEC [9] memory form factor.

7. Related Work

LSM-tree Compaction Acceleration. To alleviate the severe write amplification and compaction overhead in LSM-

tree-based key-value stores, In-Storage Computing (ISC) architectures, where the storage directly performs computations, have been proposed [10], [18]. Notably, a time-aware NDP architecture that offloads compaction operations to the SSD controller was proposed, followed by an advanced architecture applying asynchronous compaction acceleration [20], [21]. However, embedded processors (e.g., ARM [2] cores) within storage controllers have significantly lower computing power than host CPUs. Furthermore, their internal DRAM capacity is limited, and thermal issues remain unresolved.

Near-Data Processing. Recently, to overcome memory bandwidth limitations, active research has been conducted on DIMM-based PNM, which integrates computing capabilities while being compatible with standard DIMM slots. Specifically, RecNMP [12] and AxDIMM [13] are representative examples that successfully apply PNM to accelerate Personalized Recommendation Models [1], [8]. Recommendation models perform gather-reduce operations by sporadically reading sparse data. Since this is a typical memory-bound workload, it is highly advantageous to perform computations near the memory where the data resides.

8. Conclusion

In this paper, we proposed a DIMM-based PNM architecture to resolve the severe memory bandwidth bottlenecks and cache pollution caused by background compaction in LSM-tree-based key-value stores. By embedding a dedicated NMP Unit featuring a hardware comparator, an FSM controller, and a cache-bypassing DMA mechanism, our architecture effectively offloads heavy compaction operations to where the data resides. The structural performance isolation of the PNM architecture reduces L2 cache misses to 35.7% of the baseline and increases effective memory bandwidth by 1.49x. Consequently, the total execution time is significantly accelerated, reduced to 73.5% of the baseline. Ultimately, this study proves that our memory-centric computing approach provides a scalable and highly efficient hardware-accelerated solution for next-generation data-intensive database systems.

References

- [1] Amazon Personalize. <https://aws.amazon.com/personalize/>. 7
- [2] Arm. <https://www.arm.com/>. 7
- [3] Michael D Atkinson, J-R Sack, Nicola Santoro, and Thomas Strothotte. Min-max heaps and generalized priority queues. *Communications of the ACM*, 29(10):996–1000, 1986. 4
- [4] Siying Dong, Andrew Kryczka, Yanqin Jin, and Michael Stumm. Rocksdb: Evolution of development priorities in a key-value store serving large-scale applications. *ACM Transactions on Storage (TOS)*, 17(4):1–32, 2021. 1
- [5] Facebook. RocksDB. <http://rocksdb.org/>, 2015. 1
- [6] Facebook. RocksDB. <https://github.com/facebook/rocksdb>, 2025. GitHub repository, Version 9.11.1. 5
- [7] Google. LevelDB. <https://github.com/google/leveldb>, 2014. 1
- [8] Google Cloud Platform. <https://cloud.google.com/solutions/recommendations-using-machine-learning-on-compute-engine>. 7
- [9] JEDEC. <https://www.jedec.org/>. 6
- [10] Ipoom Jeong, Jinghan Huang, Chuxuan Hu, Dohyun Park, Jaeyoung Kang, Nam Sung Kim, and Yongjoo Park. Upp: Universal predicate pushdown to smart storage. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, pages 419–433, 2025. 7
- [11] Michael I Jordan and Tom M Mitchell. Machine learning: Trends, perspectives, and prospects. *Science*, 349(6245):255–260, 2015. 2
- [12] Liu Ke, Udit Gupta, Benjamin Youngjae Cho, David Brooks, Vikas Chandra, Utku Diril, Amin Firoozshahian, Kim Hazelwood, Bill Jia, Hsien-Hsin S Lee, et al. Recnmp: Accelerating personalized recommendation with near-memory processing. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, pages 790–803. IEEE, 2020. 2, 7
- [13] Liu Ke, Xuan Zhang, Jinin So, Jong-Geon Lee, Shin-Haeng Kang, Sukhan Lee, Songyi Han, YeonGon Cho, Jin Hyun Kim, Yongsuk Kwon, et al. Near-memory processing in action: Accelerating personalized recommendation with axdim. *IEEE Micro*, 42(1):116–127, 2021. 2, 7
- [14] Hyeontaek Lim, David G Andersen, and Michael Kaminsky. Towards accurate and fast evaluation of {Multi-Stage} log-structured designs. In *14th USENIX Conference on File and Storage Technologies (FAST 16)*, pages 149–166, 2016. 1
- [15] Jason Lowe-Power, Abdul Mutaal Ahmad, Ayaz Akram, Mohammad Alian, Rico Amslinger, Matteo Andreozzi, Adrià Armejach, Nils Asmussen, Brad Beckmann, Srikanth Bharadwaj, Gabe Black, Gedare Bloom, Bobby R. Bruce, Daniel Rodrigues Carvalho, Jeronimo Castrillon, Lizhong Chen, Nicolas Derumigny, Stephan Diestelhorst, Wendy El-sasser, Carlos Escuin, Marjan Fariborz, Amin Farmahini-Farahani, Pouya Fotouhi, Ryan Gambord, Jayneel Gandhi, Dibakar Gope, Thomas Grass, Anthony Gutierrez, Bagus Hanindhito, Andreas Hansson, Swapnil Haria, Austin Harris, Timothy Hayes, Adrian Herrera, Matthew Horsnell, Syed Ali Raza Jafri, Radhika Jagtap, Hanhwi Jang, Reiley Jeyapaul, Timothy M. Jones, Matthias Jung, Subash Kan-noth, Hamidreza Khaleghzadeh, Yuetsu Kodama, Tushar Krishna, Tommaso Marinelli, Christian Menard, Andrea Mondelli, Miquel Moreto, Tiago Mück, Omar Naji, Krishnendra Nathella, Hoa Nguyen, Nikos Nikoleris, Lena E. Olson, Marc Orr, Binh Pham, Pablo Prieto, Trivikram Reddy, Alec Roelke, Mahyar Samani, Andreas Sandberg, Javier Setoain, Boris Shingarov, Matthew D. Sinclair, Tuan Ta, Rahul Thakur, Giacomo Travaglini, Michael Upton, Nilay Vaish, Ilias Vougioukas, William Wang, Zhengrong Wang, Norbert Wehn, Christian Weis, David A. Wood, Hongil Yoon, and Éder F. Zulian. The gem5 simulator: Version 20.0+, 2020. 1

- [16] Sam Madden. From databases to big data. *IEEE Internet Computing*, 16(3):4–6, 2012. 2
- [17] Patrick O’Neil, Edward Cheng, Dieter Gawlick, and Elizabeth O’Neil. The log-structured merge-tree (lsm-tree). *Acta informatica*, 33(4):351–385, 1996. 1
- [18] Zhenyuan Ruan, Tong He, and Jason Cong. {INSIDER}: Designing {In-Storage} computing system for emerging {High-Performance} drive. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 379–394, 2019. 7
- [19] Seref Sagiroglu and Duygu Sinanc. Big data: A review. In *2013 international conference on collaboration technologies and systems (CTS)*, pages 42–47. IEEE, 2013. 2
- [20] Hui Sun, Wei Liu, Jianzhong Huang, Song Fu, Zhi Qiao, and Weisong Shi. Near-data processing-enabled and time-aware compaction optimization for lsm-tree-based key-value stores. In *Proceedings of the 48th International Conference on Parallel Processing*, pages 1–11, 2019. 7
- [21] Hui Sun, Bendong Lou, Chao Zhao, Deyan Kong, Chaowei Zhang, Jianzhong Huang, Yinliang Yue, and Xiao Qin. Asynchronous compaction acceleration scheme for near-data processing-enabled lsm-tree-based kv stores. *ACM Transactions on Embedded Computing Systems*, 23(6):1–33, 2024. 7
- [22] Zhi-Hua Zhou. *Machine learning*. Springer nature, 2021. 2